

**МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ**

**ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«ДОНСКОЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»
(ДГТУ)**

КОНСПЕКТ ЛЕКЦИИ № 2

АСИМПТОТИЧЕСКИЙ АНАЛИЗ АЛГОРИТМОВ. РЕКУРРЕНТНОСТИ И ЭКСПЕРИМЕНТАЛЬНАЯ ПРОВЕРКА

по дисциплине «Алгоритмы и структуры данных»

Направление подготовки: 03.03.01 Прикладные математика и физика
Направленность (профиль): 03.03.01 Прикладные физика и математика
Семестр: 7 · Объем лекции: 2 академических часа
Компетенция: ПК-4 · Индикатор: ПК-4.4

Ростов-на-Дону
2026

1. Паспорт лекции

Позиция	Содержание
Место в РПД	Раздел 1. Основы алгоритмизации и анализа сложности; вторая из двух лекций раздела.
Цель	Научить формально и экспериментально оценивать ресурсоемкость алгоритма, различать асимптотические границы и обоснованно интерпретировать результаты измерений.
Планируемый результат	Обучающийся задает размер входа и функцию трудоемкости, применяет O -, Ω - и Θ -оценки, анализирует лучший, средний и худший случаи, решает типовые рекуррентности и проектирует воспроизводимый вычислительный эксперимент.
Связь с ПК-4.4	Использует современные программные средства для реализации, инструментирования, профилирования и сравнения алгоритмов, связывая измерения с теоретической моделью стоимости.
Предварительная подготовка	Лекция № 1: спецификация, RAM-модель, корректность, инвариант и вариант цикла; базовые функции, суммы и логарифмы.
Форма текущего контроля	Устный анализ фрагментов псевдокода, мини-задача на асимптотические границы, интерпретация графика и домашний вычислительный эксперимент.

2. Планируемые результаты по уровням

Уровень	После лекции обучающийся демонстрирует
Базовый	Определяет размер входа; различает O , Ω и Θ ; распознает стандартные классы роста; подсчитывает число операций в простом цикле.
Средний	Выводит оценки лучшего, среднего и худшего случаев; решает типовые рекуррентности разворачиванием или деревом рекурсии; сопоставляет теоретическую оценку и измеренное время.
Продвинутый	Выбирает адекватную модель стоимости для физико-математических данных; обосновывает вероятностную модель среднего случая; проектирует воспроизводимый эксперимент и объясняет отклонения от асимптотического прогноза.

3. Рекомендуемая организация 90 минут

Этап	Время	Деятельность
Проблемная постановка	0–7 мин	Почему время в секундах на одном компьютере не является свойством алгоритма.
Функция трудоемкости	7–20 мин	Размер входа, элементарная операция, точный подсчет и случаи трудоемкости.
Асимптотические оценки	20–42 мин	Формальные определения O , Ω , Θ ; правила и классы роста.
Сквозной пример	42–55 мин	FIRST-THRESHOLD: лучший, худший и средний случаи.
Рекуррентности	55–72 мин	Развертывание, дерево рекурсии, идея метода подстановки; бинарный поиск и сортировка слиянием.
Эксперимент	72–84 мин	План измерений, повторяемость, медиана, логарифмический график и интерпретация.
Закрепление	84–90 мин	Мини-задача, контрольные вопросы и постановка домашнего задания.

4. Понятийный аппарат

Понятие	Рабочее определение
---------	---------------------

Понятие	Рабочее определение
Размер входа n	Числовой параметр, отражающий объем данных, существенный для стоимости алгоритма: число элементов, вершин и ребер, битов или отсчетов.
Трудоемкость $T_A(x)$	Число выбранных элементарных операций алгоритма A на конкретном входе x в зафиксированной модели вычислений.
Ресурсная функция	Зависимость потребления времени или памяти от размера входа; может задаваться для лучшего, худшего или среднего случая.
Асимптотическая граница	Сравнение функций при достаточно больших n с точностью до постоянных множителей.
Рекуррентность	Соотношение, выражающее стоимость задачи размера n через стоимости подзадач меньшего размера и работу вне рекурсивных вызовов.
Вычислительный эксперимент	Воспроизводимая серия измерений по заранее заданному протоколу, предназначенная для проверки следствий теоретической модели.

5. Развернутый сценарий лекции по презентации

Ниже приведен связный сценарий выступления, согласованный с последовательностью 26 слайдов презентации. Текст рассчитан на объяснение материала вслух: определения вводятся после проблемных ситуаций, формулы сразу связываются с конкретными данными, а каждый смысловой блок заканчивается переходом к следующему.

Слайд 1. Тема и место лекции в курсе

На первой лекции мы обсуждали, что алгоритм задается не только последовательностью команд. Для него важны предусловие, постусловие, инвариант и доказательство завершения. Теперь возникает следующий вопрос: даже если два алгоритма корректны, какой из них позволит обработать больший объем данных за приемлемое время?

Эта лекция вводит три взаимосвязанных инструмента. Асимптотический анализ описывает, как растет потребность в ресурсах при увеличении входа. Рекуррентности позволяют увидеть стоимость рекурсивных вызовов. Вычислительный эксперимент проверяет, проявляется ли теоретическая картина в конкретной реализации.

Демонстрационный пример. Навигатор может предложить два правильных маршрута. Оба приводят к месту назначения, но один удобен для короткой поездки, а другой лучше масштабируется при пробках и дальнем расстоянии. С алгоритмами ситуация похожа: корректность еще не отвечает на вопрос о практической пригодности.

Вопрос аудитории. Какие ресурсы программы, кроме времени, могут стать ограничением?

Переход. Начнем с сомнения, которое часто появляется после первого запуска программы: можно ли судить об алгоритме по секундам на экране?

Слайд 2. Почему два запуска нельзя сравнивать напрямую

Представим две реализации одной задачи. Программа В завершилась за 0,8 секунды, а программа А – за 1,1 секунды. Само по себе это наблюдение не доказывает превосходство алгоритма В. Разница могла возникнуть из-за процессора, версии интерпретатора, фоновой загрузки, прогрева среды, расположения данных в кэше или особенностей входа.

Нужно разделить четыре уровня: идею алгоритма, способ программной реализации, среду выполнения и конкретные данные. Асимптотический анализ в первую очередь

сравнивает идеи в выбранной модели стоимости. Эксперимент затем проверяет реализации в одинаковых и документированных условиях.

Демонстрационный пример. В магазине одна очередь продвинулась быстрее другой. Нельзя сразу заключить, что второй кассир работает эффективнее: в первой тележке могло быть сорок товаров, требовалась проверка возраста или произошла замена чека. Для честного сравнения нужны сопоставимые покупатели и несколько наблюдений.

1. Зафиксировать одинаковую задачу и одинаковые входные данные.
2. Запустить реализации в одной среде и повторить измерение много раз.
3. Одновременно подсчитать ключевые операции, чтобы отделить алгоритм от скорости компьютера.
4. Сформулировать вывод только для проверенного диапазона размеров.

Вопрос аудитории. Какие из четырех факторов – алгоритм, реализация, среда, данные – изменятся, если ту же программу запустить на ноутбуке от батарей?

Переход. *Чтобы измерение стало осмысленным, сначала уточним, чему именно студент должен научиться к концу лекции.*

Слайд 3. Результаты обучения

После лекции студент должен уметь не просто назвать обозначения O , Ω и Θ , а применить их к функции стоимости. Вторая группа умений связана с рекурсией: по структуре вызовов нужно составить рекуррентность и объяснить происхождение каждого слагаемого. Третья группа относится к эксперименту: сформулировать гипотезу, выбрать размеры, выполнить серию повторов и интерпретировать результат.

Эти действия непосредственно раскрывают индикатор ПК-4.4. Современное программное средство используется не само по себе, а для реализации, инструментирования, профилирования и проверки алгоритмического решения.

Вопрос аудитории. Какой результат труднее проверить: знание определения Θ или умение объяснить расхождение теории и измерения?

Переход. *Объединим ожидаемые результаты в последовательность вопросов, по которой будет построена вся лекция.*

Слайд 4. Логика лекции от подсчета к проверке

Мы будем двигаться по трем вопросам. Первый: что считаем? Здесь выбираются размер входа и элементарная операция. Второй: как растет стоимость? Здесь различаются случаи и вводятся асимптотические границы. Третий: совпало ли предсказание с поведением программы? На него отвечают рекуррентности и воспроизводимый эксперимент.

Порядок важен. Если размер входа не определен, невозможно построить функцию роста. Если не определена модель стоимости, нельзя объяснить, почему один шаг считается постоянным. Если нет гипотезы, серия измерений превращается в набор чисел без критерия интерпретации.

Демонстрационный пример. При планировании поездки вопрос «сколько займет дорога» требует уточнить расстояние, вид транспорта и условия движения. Ответ для пяти километров пешком нельзя механически переносить на пятьсот километров.

Переход. *Теперь разведем роли математической модели и наблюдаемой программы, чтобы не требовать от каждой больше, чем она может дать.*

Слайд 5. Теоретическая модель и наблюдаемая реализация

Теоретическая функция $T_A(x)$ подсчитывает выбранные операции алгоритма A на конкретном входе x . Она отвечает на вопрос о структуре вычисления. Измеренное время отвечает на другой вопрос: сколько заняла эта реализация в этой среде на этих данных. Между ними находится предсказание: если число операций растет как n^2 , то при достаточно больших n время обычно должно проявлять близкий характер роста.

Слово «обычно» здесь требует осторожности. Операции могут иметь разную фактическую стоимость, система памяти работает ступенчато, а оптимизатор меняет машинный код. Поэтому эксперимент проверяет следствия модели, но не подменяет математическое доказательство.

Демонстрационный пример. Рецепт указывает количество действий и продуктов, а фактическое время приготовления зависит от плиты, ножа и опыта человека. Тем не менее удвоение числа порций обычно увеличивает объем работы. Рецепт играет роль модели, кухня - среды, а приготовленное блюдо - результата реализации.

Переход. Чтобы перейти от конкретного входа x к функции $T(n)$, необходимо определить, что означает размер входа n .

Слайд 6. Как выбрать размер входа

Для массива естественный размер - число элементов или отсчетов. Для графа часто нужны два параметра: n вершин и m ребер. Для матрицы важны обе размерности и способ хранения. Для целого числа сложность алгоритма обработки определяется не самим числовым значением N , а длиной его записи, приблизительно $\log_2 N$ бит.

Параметр выбирают не по привычке, а по связи с числом существенных операций. Например, для разреженного графа оценка $O(n^2)$ может скрыть более точную зависимость $O(n+m)$. Для прямоугольной матрицы обозначение n без пояснения не позволяет понять, идет ли речь о числе строк, столбцов или всех элементов.

Демонстрационный пример. Телефонная книга из 10 000 записей имеет размер 10 000, если задача - найти фамилию. Карта дорог требует учесть и количество населенных пунктов, и число дорог. PIN-код со значением 9999 не в 9999 раз длиннее кода 0001: оба состоят из четырех символов.

Объект	Параметр размера	Почему он существен
Временной ряд	n отсчетов	Определяет число проверяемых значений
Граф датчиков	n вершин, m связей	Обход зависит от элементов и связей
Матрица $a \times b$	a, b и формат хранения	Число элементов равно $a \cdot b$
Целое N	$\lceil \log_2 N \rceil + 1$ бит	Алгоритм получает запись числа

Вопрос аудитории. Как задать размер входа для списка координат точек трехмерного пространства?

Переход. Определив размер, можно отвлечься от единичных секунд и сравнить скорость роста функций при увеличении n .

Слайд 7. Переход к асимптотике

Слайд-разделитель обозначает смену масштаба рассуждения. Нас интересует поведение функций при достаточно больших n . Константы и младшие члены по-прежнему влияют на реальное время, особенно на малых данных, но не определяют долгосрочный темп роста.

Важно не обещать, что асимптотически лучший алгоритм будет быстрее при любом n . Асимптотика отвечает на вопрос о масштабировании. Точка, после которой преимущество проявится, зависит от коэффициентов и среды.

Демонстрационный пример. Автомобиль быстрее велосипеда на дальней дистанции, но на маршруте в двести метров время запуска двигателя и парковки может сделать велосипед практичнее. Так же алгоритм с меньшим порядком роста может проигрывать на малых входах из-за больших постоянных расходов.

Переход. Для точного разговора о росте введем три вида границ одной функции.

Слайд 8. Границы O , Ω и Θ

Верхняя асимптотическая граница $O(g(n))$ означает, что после некоторого n_0 функция $f(n)$ не превосходит $c_2 g(n)$. Нижняя граница $\Omega(g(n))$ означает, что после некоторого порога $f(n)$ не опускается ниже $c_1 g(n)$. Если обе границы имеют один порядок, получаем $\Theta(g(n))$.

Константы c_1 , c_2 и порог n_0 являются частью доказательства. Они не обязаны быть оптимальными. Нужно лишь показать, что подходящие положительные значения существуют и не зависят от n .

Демонстрационный пример. Оценка времени доставки «не более двух часов» задает верхнюю границу, а «не менее двадцати минут» - нижнюю. Если при увеличении расстояния обе оценки пропорциональны расстоянию, длительность имеет линейный порядок роста, хотя точное число минут заранее неизвестно.

Вопрос аудитории. Может ли функция $5n$ одновременно принадлежать $O(n)$ и $O(n^2)$? Что из этих утверждений точнее?

Переход. Разберем самую частую ошибку: превращение O в знак точного равенства или в синоним худшего случая.

Слайд 9. Что на самом деле означает O

Запись $f(n) \in O(g(n))$ утверждает только наличие асимптотической верхней границы. Она не говорит, что функции равны, не описывает «обычный» запуск и не сообщает, какой случай рассматривается. Сначала выбирается функция - например, $T_{\text{worst}}(n)$ или $T_{\text{avg}}(n)$, - и только затем для нее доказывается граница.

Поэтому фраза «алгоритм работает за $O(n^2)$ » без контекста может быть формально верной, но слишком слабой. Линейная функция также принадлежит $O(n^2)$. Если известна точная граница, полезнее сказать $\Theta(n)$.

Демонстрационный пример. Если преподаватель сообщает, что проверка работы займет не более недели, это не означает, что она обязательно займет ровно неделю. Верхняя граница допускает более раннее завершение. Так и $O(n^2)$ может быть корректной, но неточной оценкой для алгоритма с линейной стоимостью.

Переход. Закрепим определения полноценным доказательством, в котором явно выберем константы.

Слайд 10. Пошаговое доказательство квадратичного порядка

Рассмотрим $f(n)=3n^2+20n+8$. Чтобы доказать $\Theta(n^2)$, нужны две части. Для верхней границы при $n \geq 1$ используем неравенства $n \leq n^2$ и $1 \leq n^2$. Тогда $20n \leq 20n^2$, а $8 \leq 8n^2$. Складывая оценки, получаем $f(n) \leq 31n^2$. Подходят $c_2=31$ и $n_0=1$.

Для нижней границы достаточно оставить положительный старший член: $f(n) \geq 3n^2$. Подходят $c_1=3$ и тот же $n_0=1$. Обе границы доказаны, поэтому $f(n) \in \Theta(n^2)$. Младшие члены не были отброшены без объяснения; их вклад был ограничен через n^2 .

1. Записать целевую функцию сравнения $g(n)=n^2$.
2. Выбрать порог $n_0=1$, после которого n и 1 можно ограничить через n^2 .
3. Получить верхнее неравенство $f(n)\leq 31n^2$.
4. Получить нижнее неравенство $f(n)\geq 3n^2$.
5. Сделать вывод о пересечении $O(n^2)$ и $\Omega(n^2)$.

Вопрос аудитории. Можно ли выбрать для верхней границы $c_2=40$? Изменится ли от этого класс Θ ?

Переход. Теперь посмотрим, почему различия между классами становятся особенно заметны при увеличении масштаба.

Слайд 11. Как расходятся классы роста

На небольших n линии разных классов могут находиться близко и даже пересекаться. При росте n характер меняется. Постоянная стоимость почти не реагирует на размер; логарифмическая увеличивается медленно; линейная удваивается при удвоении n ; квадратичная возрастает примерно в четыре раза.

Особенно важно различие между $n \log_2 n$ и n^2 . При $n=1024$ получаем около 10 240 условных операций против 1 048 576. Разница уже превышает два порядка, хотя для $n=8$ значения 24 и 64 кажутся сопоставимыми.

Демонстрационный пример. При сортировке небольшой стопки карточек простой метод может быть достаточен. При обработке миллиона измерений квадратичное сравнение всех пар становится недостижимым, тогда как $n \log n$ остается практически выполнимым.

n	$\log_2 n$	n	$n \log_2 n$	n^2
8	3	8	24	64
64	6	64	384	4 096
1 024	10	1 024	10 240	1 048 576
1 000 000	≈ 20	1 000 000	≈ 20 млн	10^{12}

Переход. Один и тот же алгоритм, однако, может выполнять разное число шагов на разных входах одинакового размера. Поэтому вводятся случаи трудоемкости.

Слайд 12. Лучший и худший случаи

Зафиксируем все допустимые входы размера n . Лучший случай - минимальная стоимость среди них. Худший случай - максимальная. Худшая оценка дает гарантию для любого входа, а лучшая показывает лишь возможность удачного сценария. Эти функции относятся к алгоритму и выбранной модели операций.

Нельзя выбирать удобный вход после измерения и называть его характеристикой всего алгоритма. Для инженерной системы худший случай особенно важен, когда превышение времени реакции недопустимо.

Демонстрационный пример. Ищем ключ в карманах. Лучший случай - ключ лежит в первом проверенном кармане. Худший - ключ находится в последнем или его нет. Число карманов одинаково, но расположение ключа меняет стоимость поиска.

Вопрос аудитории. Какой случай важнее для системы аварийного оповещения, где ответ должен быть получен не позднее заданного срока?

Переход. Между возможностью лучшего и гарантией худшего хочется описать типичное поведение. Для этого потребуется вероятностная модель.

Слайд 13. Почему средний случай требует распределения

Средняя стоимость - математическое ожидание случайной величины $T_A(X)$. Поэтому сначала нужно определить, с какой вероятностью возникают разные входы. Равномерное распределение удобно для вычислений, но физические данные часто устроены иначе: пороговые события могут чаще появляться в начале серии, группироваться или вовсе отсутствовать.

Если распределение неизвестно, корректнее говорить об эмпирической средней на конкретной выборке и описать, как она собрана. Нельзя без проверки объявлять эту выборку моделью будущих данных.

Демонстрационный пример. Время ожидания автобуса зависит от момента прихода пассажира. При равномерном приходе между рейсами среднее ожидание равно половине интервала. Но утром пассажиры приходят волнами перед началом занятий, и равномерная модель может давать неверный прогноз.

Переход. Применим различие случаев к единому сквозному алгоритму поиска первого достижения порога.

Слайд 14. FIRST THRESHOLD и три сценария

Алгоритм FIRST-THRESHOLD последовательно просматривает значения $x[i]$ и останавливается, когда встречает первое значение не меньше порога τ . Обозначим через k число просмотренных элементов. Стоимость линейно зависит от k : $T(n,k)=a+b \cdot k$, поэтому $T(n,k) \in \Theta(k)$.

Для лучшего случая $k=1$. Для худшего $k=n$. Если позиция остановки равновероятна среди $1, \dots, n$, то $E[k]=(1+2+\dots+n)/n=(n+1)/2$, то есть средний порядок остается линейным. Коэффициент примерно вдвое меньше худшего, но класс $\Theta(n)$ не меняется.

Демонстрационный пример. Пусть температурные измерения равны [18,2; 18,7; 19,4; 21,1; 22,0], а порог $\tau=20,0$. Алгоритм проверяет четыре элемента и возвращает индекс 3 при нумерации с нуля.

1. $i=0$: $18,2 < 20,0$, поэтому увеличиваем i .
2. $i=1$: $18,7 < 20,0$, продолжаем.
3. $i=2$: $19,4 < 20,0$, продолжаем.
4. $i=3$: $21,1 \geq 20,0$, цикл прекращается.
5. Возвращаем 3; просмотрено $k=4$ элемента.

Псевдокод сквозного примера

```
FIRST-THRESHOLD( $x, n, \tau$ )
   $i \leftarrow 0$ 
  while  $i < n$  and  $x[i] < \tau$ 
     $i \leftarrow i + 1$ 
  if  $i = n$ 
    return -1
  return  $i$ 
```

Вопрос аудитории. Как изменится ожидаемое k , если превышение порога с большой вероятностью появляется в первых десяти отсчетах независимо от общего n ?

Переход. Для циклического алгоритма стоимость получилась напрямую. Рекурсивные алгоритмы удобнее описывать уравнениями, связывающими задачу с подзадачами.

Слайд 15. Переход к рекуррентностям

Рекуррентность отражает устройство рекурсивного алгоритма. В ней нужно учесть число вызовов, размеры подзадач, работу вне рекурсии и базовый случай. Если хотя бы один элемент пропущен, уравнение перестает соответствовать программе.

Полезно каждый раз проговаривать происхождение слагаемых. Запись $T(n)=2T(n/2)+cn$ означает: создаются две подзадачи половинного размера, а разделение или объединение результатов требует линейной работы cn .

Демонстрационный пример. Разделение стопки документов между двумя исполнителями напоминает рекурсию. Нужно учесть время каждого исполнителя и дополнительную работу по разделению стопки и объединению результатов.

Переход. Первый пример рекуррентности возникает в бинарном поиске, где после сравнения остается только половина области.

Слайд 16. Бинарный поиск

Для отсортированного массива бинарный поиск сравнивает ключ со средним элементом и продолжает работу только в одной половине. Поэтому $T(n)=T(\lfloor n/2 \rfloor)+c$, где c описывает сравнение и вычисление новой границы. Базовый случай имеет постоянную стоимость.

После k шагов размер области не превосходит $n/2^k$. Остановка происходит, когда $n/2^k \leq 1$. Отсюда $2^k \geq n$ и $k \geq \log_2 n$. Число уровней равно $\Theta(\log n)$, а работа каждого уровня постоянна, поэтому $T(n) \in \Theta(\log n)$.

Демонстрационный пример. Ищем число 73 в отсортированном массиве из 16 значений. После первого сравнения остается не более 8 элементов, затем 4, 2 и 1. Для любого ключа достаточно примерно четырех делений области.

1. Область содержит 16 элементов; выбираем середину.
2. После сравнения оставляем нужную половину из 8 элементов.
3. Повторяем для 4, затем 2 и 1 элемента.
4. Получаем не более $\log_2 16=4$ сокращений области.

Вопрос аудитории. Почему бинарный поиск нельзя без изменения применить к неотсортированному массиву?

Переход. В бинарном поиске на каждом уровне остается одна ветвь. В сортировке слиянием сохраняются обе половины, поэтому дерево рекурсии устроено иначе.

Слайд 17. Сортировка слиянием и дерево рекурсии

Сортировка слиянием делит массив на две половины, рекурсивно сортирует каждую и линейно сливает результаты. Рекуррентность имеет вид $T(n)=2T(n/2)+cn$. На первом уровне выполняется слияние объема n . На следующем уровне два слияния по $n/2$ снова дают суммарно n . На любом уровне сумма размеров подзадач равна n .

Глубина дерева равна $\log_2 n$, а работа уровня - $\Theta(n)$. Умножение дает $\Theta(n \log n)$. На уровне листьев есть n подзадач размера 1 с постоянной работой; их общий вклад $\Theta(n)$ не меняет итоговый класс.

Демонстрационный пример. Для массива [8, 3, 6, 2, 7, 1, 5, 4] сначала образуются половины по четыре элемента, затем пары и одиночные значения. При обратном движении сливаются пары, четверки и весь массив. На каждом этапе слияния каждый элемент участвует в ограниченном числе сравнений.

Уровень	Число блоков	Размер блока	Суммарная работа
---------	--------------	--------------	------------------

Уровень	Число блоков	Размер блока	Суммарная работа
0	1	n	cn
1	2	n/2	$2 \cdot c(n/2) = cn$
2	4	n/4	$4 \cdot c(n/4) = cn$
$\log_2 n$	n	1	$\Theta(n)$

Переход. Два примера показывают, что одну рекуррентность удобно развернуть, а другую - представить деревом. Систематизируем методы решения.

Слайд 18. Методы решения рекуррентностей

Развертывание несколько раз подставляет рекуррентность саму в себя и выявляет закономерность. Дерево рекурсии показывает число подзадач и работу уровней. Метод подстановки начинается с гипотезы о границе и подтверждает ее индукцией. Основная теорема дает быстрый результат для стандартной формы $aT(n/b) + f(n)$, но только после проверки условий.

Метод выбирают по структуре уравнения, а не по привычке. В учебном решении нужно сначала объяснить, откуда появилась рекуррентность, затем решить ее и лишь после этого интерпретировать результат для алгоритма.

Рекуррентность	Удобный метод	Результат
$T(n) = T(n/2) + c$	Развертывание	$\Theta(\log n)$
$T(n) = 2T(n/2) + cn$	Дерево рекурсии	$\Theta(n \log n)$
$T(n) = T(n-1) + n$	Развертывание суммы	$\Theta(n^2)$
$T(n) = 3T(n/2) + n$	Основная теорема после проверки	$\Theta(n^{\log_2 3})$

Вопрос аудитории. Почему нельзя применять основную теорему к любой рекуррентности, в которой встречается n?

Переход. Математическая оценка сформулирована. Теперь проверим, видны ли ее следствия в работающей программе.

Слайд 19. Переход к вычислительному эксперименту

Эксперимент выполняет две функции. Он обнаруживает ошибки реализации и показывает диапазон, в котором теоретическая модель объясняет наблюдаемое поведение. Однако конечный набор измерений не доказывает Θ -границу для всех достаточно больших n.

Поэтому сильный экспериментальный вывод звучит локально: «в диапазоне n от ... до ... отношение и наклон согласуются с гипотезой $\Theta(n^2)$ ». Формулировка «эксперимент доказал квадратичную сложность» некорректна.

Демонстрационный пример. Несколько измерений температуры могут подтвердить, что датчик работает ожидаемо в проверенном диапазоне, но не доказывают его поведение при любой возможной температуре и в любой среде.

Переход. Чтобы набор измерений стал аргументом, до запуска нужно сформулировать гипотезу и протокол.

Слайд 20. Как построить воспроизводимый эксперимент

Сначала записывают гипотезу: что измеряется и какой класс роста ожидается. Затем выбирают геометрическую серию размеров, например 2^{10} , 2^{11} , ..., 2^{20} . Для каждого размера готовят входы вне измеряемого участка, выполняют прогрев и серию повторов. Сохраняют все результаты, а не только удобное значение.

В качестве центральной оценки часто используют медиану: единичная фоновая задержка меньше влияет на нее, чем на среднее арифметическое. Разброс можно описывать межквартильным размахом. В протоколе фиксируют версию языка, библиотеки, режим запуска, процессор, операционную систему и зерно генератора.

Демонстрационный пример. Проверяем гипотезу о линейной стоимости FIRST-THRESHOLD в худшем случае. Для каждого n создаем массив, где порог не достигается, подсчитываем обращения и измеряем время. Счетчик должен расти точно пропорционально n ; время может колебаться, но его отношение при удвоении n должно стремиться к 2 на достаточно больших входах.

1. Записать гипотезу до просмотра результатов.
2. Подготовить одинаково устроенные входы для всех размеров.
3. Отделить прогрев и генерацию данных от измерения.
4. Выполнить не менее 15 повторов и сохранить сырые значения.
5. Сравнить счетчик операций, медиану времени и разброс.
6. Сформулировать область применимости и возможные причины отклонений.

Переход. Один из самых понятных диагностических показателей - отношение времени при удвоении размера входа.

Слайд 21. Диагностика по отношению $T(2n)$ к $T(n)$

Для линейной функции отношение $T(2n)/T(n)$ стремится к 2. Для квадратичной - к 4. Для $n \log n$ оно немного больше 2 и постепенно приближается к 2, потому что $\log(2n) = \log n + 1$. Это не доказательство класса, а удобный способ проверить согласованность серии с гипотезой.

Отношение имеет смысл только там, где измерения существенно превосходят шум таймера. Если $T(n)$ почти равно нулю, деление усиливает случайную ошибку. Нужно рассматривать последовательность отношений, а не одну пару.

Демонстрационный пример. Если медианы равны 2,1; 8,3; 33,1 и 132,5 мс при последовательном удвоении n , отношения близки к 4. Это поддерживает квадратичную гипотезу. Но нужно проверить еще несколько размеров и исключить попадание генерации данных в таймер.

Гипотеза	Ожидаемое отношение	Наблюдение при больших n
$\Theta(n)$	≈ 2	Стабилизация около 2
$\Theta(n \log n)$	> 2 , но постепенно ближе к 2	Медленное снижение
$\Theta(n^2)$	≈ 4	Стабилизация около 4

Переход. Чтобы отношение описывало алгоритм, проведем точную границу измеряемого участка.

Слайд 22. Граница измерения

Входные данные создаются до запуска таймера. Там же фиксируется зерно генератора и выполняется прогрев. Внутри измеряемого участка остается только вызов исследуемого алгоритма. После остановки таймера результат обязательно используется или проверяется, чтобы оптимизатор не смог удалить вычисление как ненужное.

Если алгоритм по условию задачи включает сортировку, ее нельзя искусственно вынести за таймер. Граница определяется исследовательским вопросом. Мы исключаем подготовку только тогда, когда хотим измерить отдельную операцию и явно описываем это в протоколе.

Демонстрационный пример. Если при сравнении линейного и бинарного поиска для каждого запроса внутри таймера заново сортировать массив, эксперимент измерит полный сценарий «подготовка плюс поиск». Это корректно для разового запроса, но не отвечает на вопрос о стоимости поиска в уже отсортированном справочнике.

Каркас измерительного эксперимента

```
for n in sizes:
    x = make_input(n, seed)      # до таймера
    warm_up(x)                  # прогрев
    samples = []
    repeat R times:
        t0 = high_resolution_clock()
        result = A(x)            # измеряемый участок
        t1 = high_resolution_clock()
        check(result)
        samples.append(t1 - t0)
    save(n, median(samples), samples)
```

Переход. *Ошибочная граница измерения - лишь одна из типичных причин неверного вывода. Соберем их в единую карту.*

Слайд 23. Типичные ошибки анализа

Первая группа ошибок относится к модели: размер входа не определен, элементарные операции меняются по ходу анализа, а O используется как знак равенства. Вторая связана с вероятностью: «средний случай» объявлен без распределения. Третья возникает в эксперименте: один запуск принимается за закономерность, а в таймер попадают генерация, ввод-вывод и подготовительная сортировка.

Есть и менее очевидные искажения: первые запуски медленнее из-за прогрева, изменение размера пересекает границу уровня кэша, сборщик мусора запускается в случайный момент, а слишком короткая операция сравнима с точностью таймера. Каждая причина требует отдельного контроля, а не удаления «неудобных» результатов.

Демонстрационный пример. Студент получил времена 0,00; 0,01 и 0,00 мс и заключил, что алгоритм имеет постоянную сложность. На самом деле длительность ниже разрешения измерителя. Нужно увеличить n или многократно повторить операцию внутри одного измерительного пакета, сохранив смысл эксперимента.

Вопрос аудитории. Какая ошибка возникнет, если для одного n использовать уже отсортированный массив, а для другого - случайный?

Переход. *Проверим, как работает диагностика по удвоению, на короткой задаче для всей аудитории.*

Слайд 24. Мини-задача по распознаванию роста

Дано наблюдение: при удвоении n медианное время увеличивается примерно в 3,8-4,2 раза. Из трех гипотез $\Theta(n)$, $\Theta(n \log n)$ и $\Theta(n^2)$ наиболее согласована с данными квадратичная. Линейный рост дал бы коэффициент около 2, а $n \log n$ - немного больше 2.

Ответ должен оставаться гипотезой. Следует измерить дополнительные размеры, посмотреть стабильность отношения, проверить счетчик операций и убедиться, что режим входов не меняется. Полезен график $\log T$ от $\log n$: его наклон для степенной зависимости $T \approx c n^p$ оценивает показатель p .

Демонстрационный пример. Пусть времена для n , $2n$ и $4n$ равны 5,0; 19,6 и 80,1 мс. Отношения 3,92 и 4,09 поддерживают $p \approx 2$. Если следующая точка дает 2,1, это сигнал проверить шум, кэш или изменение входного сценария, а не выбирать только первые две пары.

1. Сопоставить наблюдаемое отношение с теоретическими ориентирами.
2. Проверить несколько последовательных удвоений.
3. Сравнить время со счетчиком существенных операций.
4. Проверить постоянство генератора данных и границы таймера.
5. Сформулировать вывод для конкретного диапазона n .

Переход. *Перед завершением лекции используем пять вопросов как быструю самопроверку понимания.*

Слайд 25. Самопроверка

Первый вопрос проверяет границы: O дает верхнюю оценку, а Θ - совпадающие по порядку верхнюю и нижнюю. Второй возвращает нас к вероятностной модели среднего случая. Третий требует развернуть бинарный поиск до равенства $n/2^k \approx 1$. Четвертый собирает источники искажений микробенчмарка. Пятый напоминает, что стоимость подготовки данных может изменить выбор алгоритма.

При обсуждении важно просить не однословный ответ, а короткое обоснование. Например, бинарный поиск имеет $\Theta(\log n)$ для одного запроса в отсортированном массиве, но при единственном запросе предварительная сортировка $\Theta(n \log n)$ может сделать линейный поиск выгоднее.

Вопрос аудитории. Сформулируйте одним предложением, почему экспериментальная кривая не является доказательством Θ .

Переход. *Осталось собрать все части в единую схему и связать их со следующей лекцией о представлении данных в памяти.*

Слайд 26. Итог и связь со следующей темой

Полный анализ начинается с размера входа и модели стоимости. Затем для выбранной функции различаются лучший, худший или средний случай и доказывается асимптотическая граница. Для рекурсивного алгоритма структура вызовов записывается рекуррентностью. Наконец, реализация проверяется серией воспроизводимых измерений.

На следующей лекции эти инструменты будут применены к массивам и динамическим массивам. Мы сравним стоимость доступа, вставки и расширения, а также увидим, как представление в памяти влияет на фактическое время. Именно здесь станет особенно заметно различие между худшей стоимостью одной операции и амортизированной стоимостью последовательности операций.

Демонстрационный пример. Выбор между обычным списком покупок и заранее размеченными ячейками хранения зависит не только от числа записей, но и от того, какие

действия выполняются чаще: последовательный просмотр, быстрый доступ по номеру или добавление новых элементов.

Переход. После итогового обсуждения преподаватель переходит к контрольным вопросам и формулирует домашний вычислительный эксперимент.

Контрольные вопросы используются после итогового слайда или в начале следующего занятия. Ответ считается полным, если студент называет условие применимости, приводит короткий пример и различает математическую оценку и результат измерения.

6. Контрольные вопросы

1. Почему время в секундах не является переносимой характеристикой алгоритма?
2. Как выбрать размер входа для массива, целого числа и графа?
3. Чем $T_A(x)$ отличается от $T_{\text{worst}}(n)$?
4. Почему средний случай требует распределения вероятностей?
5. Как формально читаются $O(g)$, $\Omega(g)$ и $\Theta(g)$?
6. Почему утверждение $f \in O(g)$ не означает $f \in \Theta(g)$?
7. Почему O не является синонимом худшего случая?
8. Как определить доминирующий член суммы?
9. Какая рекуррентность соответствует бинарному поиску?
10. Почему сортировка слиянием имеет работу $\Theta(n)$ на каждом уровне дерева рекурсии?
11. Что должен содержать воспроизводимый протокол измерений?
12. Почему медиана часто надежнее одного запуска и среднего арифметического?
13. Что показывает отношение $T(2n)/T(n)$?
14. Почему эксперимент не доказывает асимптотическую границу?
15. Когда бинарный поиск может оказаться невыгоден, несмотря на $\Theta(\log n)$?

7. Домашнее задание

- Добавить в реализацию FIRST-THRESHOLD счетчики сравнений и обращений к массиву. Для $n = 2^8, \dots, 2^{20}$ построить серии для лучшего, худшего и заданного среднего сценария.
- Записать гипотезы до запуска, выполнить не менее 15 повторов каждого измерения времени, сохранить медиану и межквартильный размах; генерацию данных вынести из измеряемого участка.
- Построить таблицу $T(n)$, $T(2n)/T(n)$ и график $\log T$ от $\log n$. Сопоставить наклон с теоретическим классом и объяснить не менее двух отклонений.
- Решить рекуррентности: $T(n) = T(n-1) + n$; $T(n) = 2T(n/2) + 1$; $T(n) = 3T(n/2) + n$. Указать метод и асимптотическую оценку.
- Подготовить к лекции № 3 объяснение, почему вставка в середину массива и вставка после известного узла списка имеют разную стоимость и разные расходы памяти.

8. Итоги лекции

- Функция трудоемкости определяется относительно размера входа и модели элементарных операций.
- Лучший, худший и средний случаи отвечают на разные вопросы; средний случай невозможен без вероятностной модели.

- и Ω задают верхнюю и нижнюю границы, а Θ — точный порядок роста с точностью до постоянного множителя.
- Рекуррентность отражает структуру рекурсивного алгоритма и включает стоимость работы вне подзадач.
- Эксперимент проверяет реализацию и область применимости модели, но не заменяет математического доказательства.
- При выборе прикладного решения учитывают полный поток обработки данных, а не только сложность одной операции.

9. Основная и дополнительная литература

1. Панова, Т. В. Основы алгоритмизации и программирования на языке высокого уровня Си : учебное пособие / Т. В. Панова, Н. Д. Николаева. — Санкт-Петербург : БГТУ "Военмех" им. Д.Ф. Устинова, 2015. — 176 с. — ISBN 978-5-85546-874-8. — Текст : электронный // Лань : электронно-библиотечная система. — URL: <https://e.lanbook.com/book/75168> (дата обращения: 15.09.2026). — Режим доступа: для авториз. пользователей.
2. Основы алгоритмизации и программирования : учебное пособие для СПО / А. А. Бердникова, С. Л. Иванов, А. С. Лямин, А. Д. Рейн. — 2-е изд., стер. — Санкт-Петербург : Лань, 2026. — 176 с. — ISBN 978-5-507-51209-6. — Текст : электронный // Лань : электронно-библиотечная система. — URL: <https://e.lanbook.com/book/507808> (дата обращения: 15.09.2026). — Режим доступа: для авториз. пользователей.
3. Щукарёв, И. А. Основы алгоритмизации и программирования : учебное пособие / И. А. Щукарёв. — Ульяновск : УлГТУ, 2025. — 92 с. — ISBN 978-5-9795-2540-2. — Текст : электронный // Лань : электронно-библиотечная система. — URL: <https://e.lanbook.com/book/518799> (дата обращения: 15.09.2026). — Режим доступа: для авториз. пользователей.
4. Бунаков, П. Ю. Основы алгоритмизации и программирования на языке Python : учебное пособие для вузов / П. Ю. Бунаков. — Санкт-Петербург : Лань, 2026. — 228 с. — ISBN 978-5-507-53786-0. — Текст : электронный // Лань : электронно-библиотечная система. — URL: <https://e.lanbook.com/book/511913> (дата обращения: 15.09.2026). — Режим доступа: для авториз. пользователей.
5. Бунаков, П. Ю. Основы алгоритмизации и программирования на языке Python : учебное пособие для СПО / П. Ю. Бунаков. — Санкт-Петербург : Лань, 2026. — 228 с. — ISBN 978-5-507-53787-7. — Текст : электронный // Лань : электронно-библиотечная система. — URL: <https://e.lanbook.com/book/512047> (дата обращения: 15.09.2026). — Режим доступа: для авториз. пользователей.
6. Morin P. Open Data Structures. Open textbook. Разделы об анализе времени выполнения и рекуррентностях.
7. Cormen T. H., Leiserson C. E., Rivest R. L., Stein C. Introduction to Algorithms. 4th ed. MIT Press, 2022. Chapters 2-4.
8. Skiena S. S. The Algorithm Design Manual. 3rd ed. Springer, 2020. Chapters 2-3.
9. McGeoch C. C. A Guide to Experimental Algorithmics. Cambridge University Press, 2012. Chapters 1-3.
10. Актуализированная РПД дисциплины «Алгоритмы и структуры данных» для направления 03.03.01, редакция от 06.09.2026.